Algoritmo de *Work Stealing*

Octubre/2026



Contenido

- Preámbulo
  - Mecanismos de concurrencia con hilos en Java
- Estructura del artículo
- Introducción
- Un modelo de computación multihilos
- La propiedad de “hojas ocupadas”
- Un algoritmo de robo de trabajo (work-stealing) aleatorizado
- Acceso atómico y el juego de reciclaje
- Análisis del algoritmo de robo de trabajo (work-stealing) 
- Otros artículos para revisar
- Conclusiones y recomendaciones

Conceptos para recordar

- **Modelo:** una computación multihilo es un DAG de instrucciones unitarias con arcos de continuación, creación (*spawn*) y juntura (*join*).
- **Medidas:** el trabajo T_1 es el número total de instrucciones. El *span* T_∞ es la longitud del camino más largo del DAG.
- **Cota ideal (Brent/Graham):** todo planificador *greedy* logra $T \leq T_1/P + T_\infty$.
- **Computación completamente estricta:** todo arco de juntura va de un hilo hijo a su padre.
- **Ready deque:** cada procesador trabaja por abajo (como una pila), y los ladrones roban por arriba (*top*).
- **Lema 4:** en la deque, cada hilo es padre del que está debajo, y solo el del *top* pudo avanzar desde que creó a su hijo.
- **Lema 6 (juego de reciclaje):** M solicitudes de robo aleatorias producen un retardo total esperado $\leq M$.

Glosario y notación

En el artículo	En esta presentación	Símbolo
work	trabajo	
critical-path length/span	camino crítico	
spawn Edge	arco de creación	
continue edge	arco de continuación	
join edge	arco de juntura/unión	
deque edge	arco deque	
augmented dag	Grafo extendido (DAG aumentado)	
ready deque	deque de listos	
steal attempt/round	intento de robo/ronda	
critical instruction	instrucción crítica	
delay sequence	secuencia de retraso	

P es el número de procesadores, Π es la partición de rondas, $\lg = \log_2$, y $\varepsilon > 0$ es la probabilidad de fallo tolerada.

Análisis del algoritmo de Work Stealing

Análisis del algoritmo de Work Stealing

- En esta sección, se analizará el tiempo de ejecución y el costo de comunicación al ejecutar una computación completamente estricta con el algoritmo de robo de trabajo (Work-Stealing Algorithm).
- Para cualquier computación completamente estricta con WORK T_1 y SPAN T_∞, mostramos que el tiempo de ejecución esperado con P procesadores (incluyendo la sobrecarga de planificación) es:

$$T_1/P + O(T_\infty)$$

- Además, para cualquier $\epsilon > 0$, el tiempo de ejecución con P procesadores es:

$$T_1/P + O(T_\infty + \lg P + \lg(1/\epsilon))$$

- con una probabilidad de al menos $1 - \epsilon$.

Análisis del algoritmo de Work Stealing

- También se mostrará que la comunicación total esperada durante la ejecución de una computación completamente estricta es:

$$O(PT_\infty(1 + n_d)S_{\max})$$

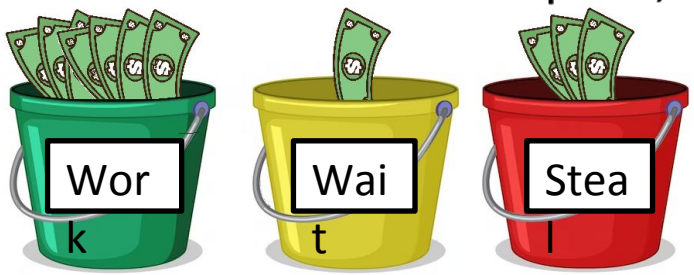
- donde n_d es el número máximo de arcos de unión (*join edges*) desde un hilo hijo hacia su hilo padre, y S_{max} es el tamaño máximo de cualquier *activation frame*.
- A diferencia del algoritmo de hojas ocupadas (*Busy-Leaves Algorithm*), el "conjunto de hilos ready" (*ready pool threads*) en el algoritmo de robo de trabajo está distribuido, por lo que no existe contención en una estructura de datos centralizada.
- No obstante, todavía es posible que haya contención cuando varios procesadores ladrones intentan robar trabajo del mismo procesador víctima al mismo tiempo.

Análisis del algoritmo de Work Stealing

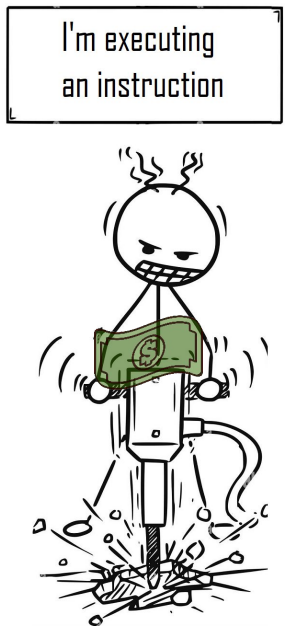
- En tal caso —como se indicó en la sección anterior— se asume conservadoramente que un adversario [término tomado prestado de teoría de juegos] encola en serie las solicitudes de robo de trabajo.
- Además, se supone que responder a una solicitud de robo de trabajo toma una unidad de tiempo.
- Esta suposición puede relajarse (por ejemplo, para permitir cualquier tiempo constante de respuesta) sin afectar de forma sustancial los resultados.

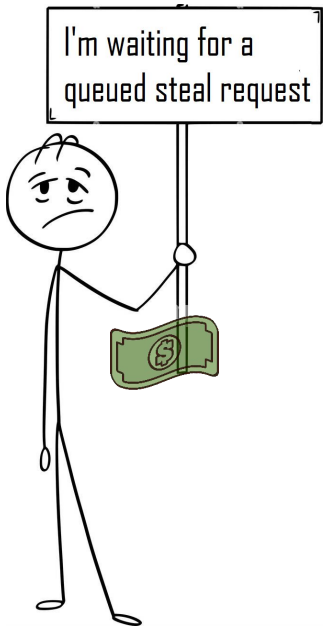
Análisis del algoritmo de Work Stealing

- Para analizar el tiempo de ejecución del algoritmo de Work Stealing ejecutando una computación multihilo completamente estricta con work T_1 y span T_∞ sobre un computador con P procesadores, se utilizará un argumento "contable" [hacer cuentas, hacer contabilidad]
- En cada paso del algoritmo, se recopilan P dólares, uno de cada procesador. En cada paso, cada procesador coloca su dólar en uno de tres baldes de acuerdo a sus acciones en dicho paso.
  - Si el procesador ejecuta una instrucción en ese paso, entonces coloca su dólar en el balde WORK.
  - Si el procesador inicia un intento de robo en ese paso, entonces coloca su dólar en el balde STEAL.
  - Si el procesador solamente espera para una petición de robo en cola en ese paso, entonces coloca su dólar en el balde WAIT.



Análisis del algoritmo de Work Stealing









Work bucket



Wait bucket



Steal bucket

¿Por qué contar dólares mide el tiempo?

- En cada paso, cada uno de los P procesadores hace exactamente una de tres cosas: ejecuta una instrucción, inicia un robo o espera.
- Por tanto, en cada paso entran exactamente P dólares.
- Si la ejecución dura T pasos, entonces

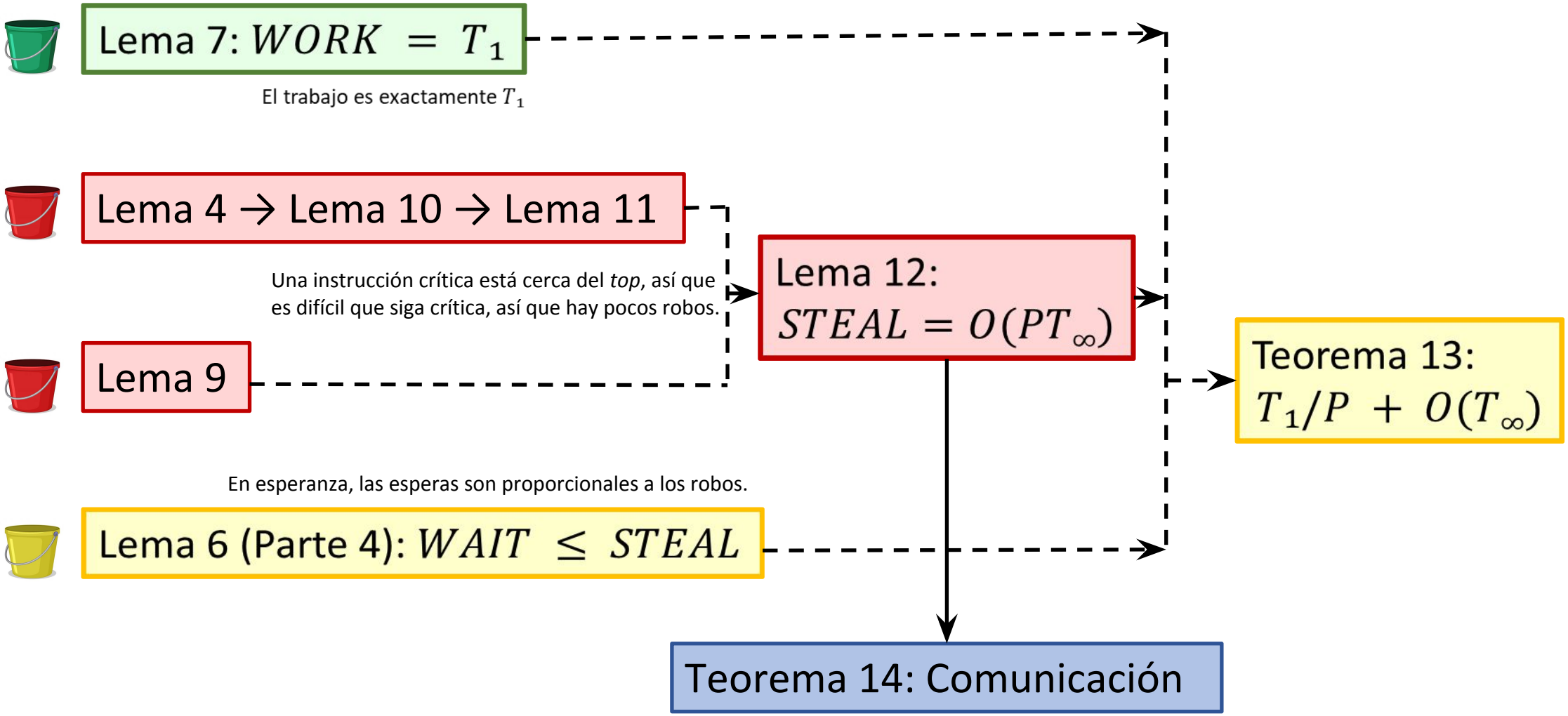
$$P \cdot T = WORK + STEAL + WAIT$$

- Despejando, $T = (WORK + STEAL + WAIT) / P$.
- Estrategia: acotar cada balde por separado y dividir por P.

$P \cdot T =$  $+$ $+$

Work bucket | Steal bucket | Wait bucket

Mapa de la demostración



Dólares en el ***work bucket***



- **Lema 7:** La ejecución de una computación multihilo completamente estricta con work T_1 utilizando el algoritmo de Work-Stealing en un computador con P procesadores termina con exactamente T_1 dólares en el balde work.
- Prueba: Un procesador coloca un dólar en el balde Work únicamente cuando ejecuta una instrucción. De este modo, como hay T_1 instrucciones en la computación, la ejecución concluye con exactamente T_1 dólares en el balde Work.

Dólares en el ***steal bucket***



- Acotar el total de dólares en el balde Steal requiere un argumento de *delay-sequence* [secuencia de retraso] mucho más complicado.
- Se introduce
  - (1) el concepto de *“ronda” de intentos work-steal* [trabajo-robo] y también se debe definir
  - (2) un dag aumentado que luego utilizaremos para definir las *instrucciones “críticas”*.

Dólares en el ***steal bucket***



- ***Idea:*** si, durante el transcurso de la ejecución, hay una gran cantidad de intentos de robos entonces se podrá identificar una secuencia de instrucciones –*delay sequence*– en el dag aumentado, de tal manera que cada uno de estos intentos de robo fue iniciado mientras alguna instrucción de la secuencia era “crítica”.
- Luego mostraremos que una instrucción “crítica” tiene poca probabilidad de que permanezca “crítica” durante una cantidad moderada de intentos de robo.
- Luego se podrá concluir que ese tipo de secuencia de retardo es poco probable que ocurra, y por tanto, para una ejecución también *será poco probable* que sea afectada por una cantidad grande de intentos de robo.

Dólares en el ***steal bucket***



- Una **Ronda** de intentos de robo son por lo menos $3P$ intentos de robo consecutivos, pero menos de $4P$ intentos consecutivos de tal manera que si un intento de robo es iniciado en el paso del instante t ocurre en una ronda en particular, entonces todos los otros intentos de robo iniciados en el mismo paso del instante t son de la misma **Ronda**.
- Podemos particionar todos los intentos de robo que ocurren durante una ejecución en rondas, de la siguiente manera:
  1. La primera ronda contiene todos los intentos de robo iniciados en los pasos de tiempo $1,2, \ldots, t_1$ donde t_1 es el menor paso de tiempo tal que se hayan iniciado al menos $3P$ intentos de robo en o antes de t_1. Decimos entonces que la primera ronda comienza en el paso de tiempo 1 y termina en el paso de tiempo t_1.

Dólares en el ***steal bucket***

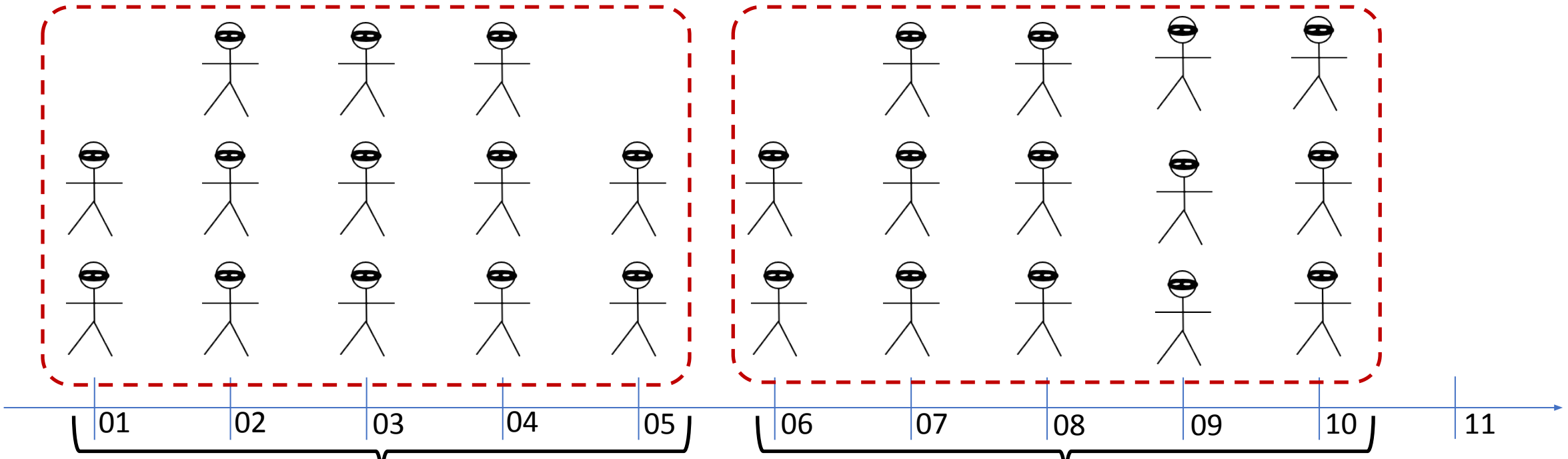


- Podemos particionar todos los intentos de robo que ocurren durante una ejecución en rondas, de la siguiente manera:
  2. En general, si la i-ésima ronda termina en el paso de tiempo t_i, entonces la (i+1)-ésima ronda comienza en el paso de tiempo $t_i + 1$ y termina en el paso de tiempo más temprano $t_{i+1} > t_i + 1$ tal que se hayan iniciado al menos $3P$ intentos de robo en los pasos de tiempo entre $t_i + 1$ y t_{i+1}, inclusive.
  3. Esos intentos de robo pertenecen a la ronda $i + 1$.
  4. Por definición, cada ronda contiene al menos $3P$ intentos de robo consecutivos. Además, dado que en un solo paso de tiempo se pueden iniciar como máximo $P - 1$ intentos de robo, cada ronda contiene menos de $4P - 1$ intentos de robo, y cada ronda dura al menos cuatro pasos de tiempo.

$$3P \leq \textbf{Ronda} < 4P$$

Entendiendo las rondas





P = 4

¿Intentos de robo ≥ 3P?

Sí, el valor es 14, es mayor o igual que 3P y menor que 4P, entonces esta es la ronda 2

Grafo extendido

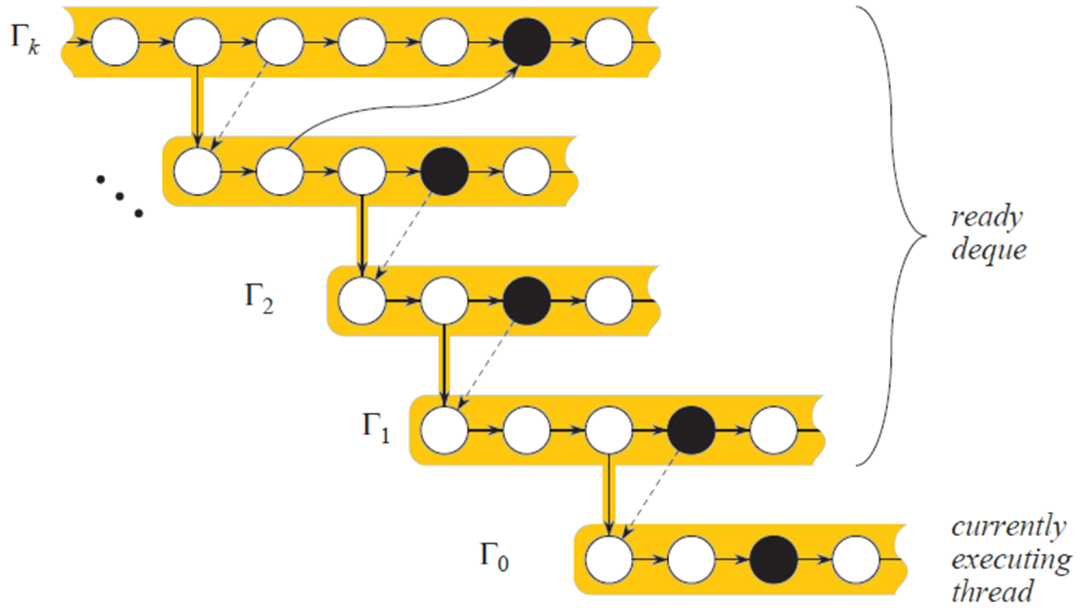


- La secuencia de instrucciones que conforma la secuencia de retraso se define con respecto a un DAG aumentado, obtenido a partir de una ligera modificación del DAG original.
- Sea G el DAG original, es decir, el grafo dirigido acíclico que contiene como vértices las instrucciones de la computación y como arcos sus enlaces de tipo continuación, creación y juntura.
- El DAG aumentado G' es el DAG original G más algunos nuevos arcos, definidos del siguiente modo:
  - Para cada conjunto de instrucciones u, v y w tal que (u,v) es un arco de creación y (u,w) es un arco de continuación, se añade en G' un arco deque (w, v).

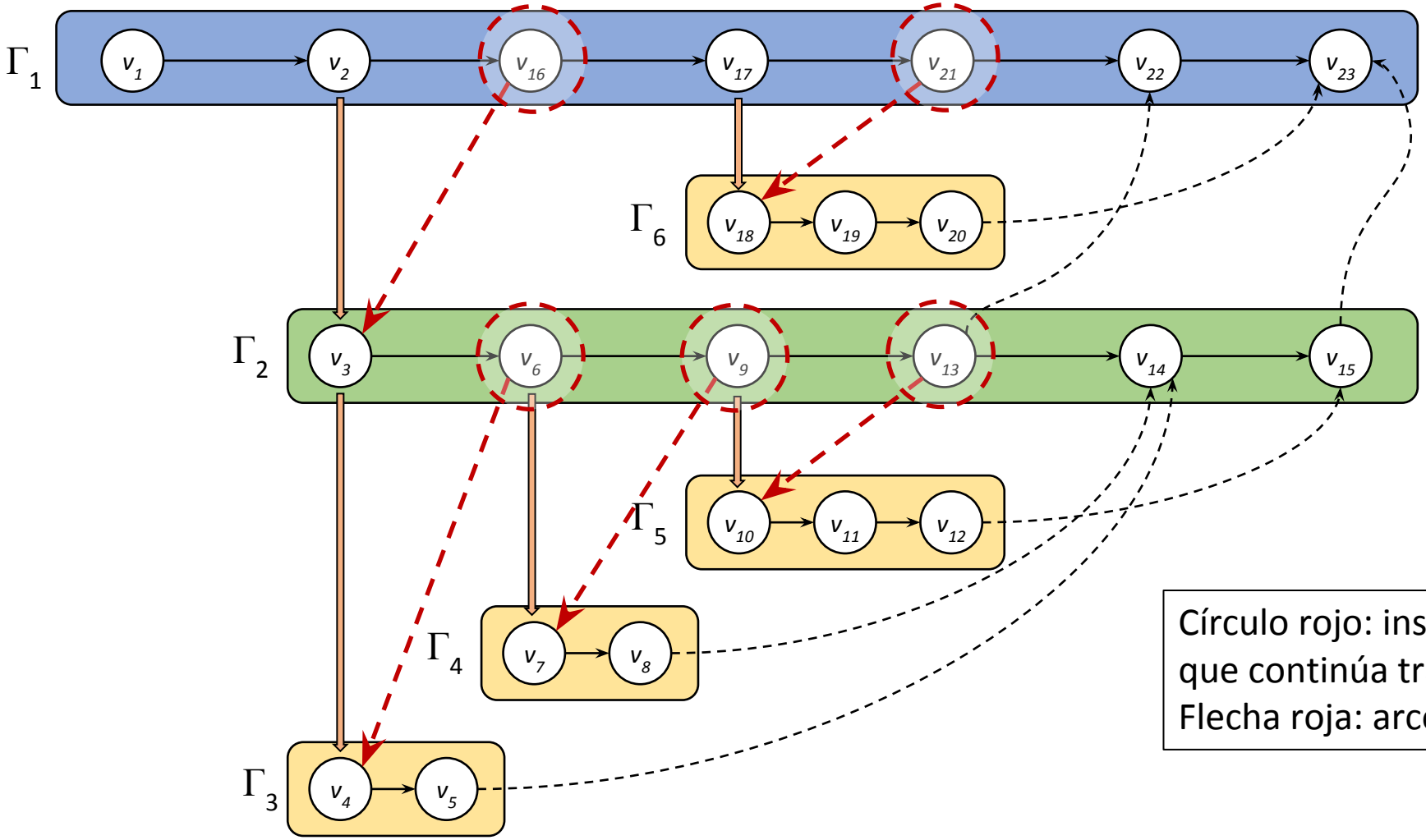
Grafo extendido



- Estos arcos deque se muestran como líneas discontinuas.
- En la Sección 2 se hizo la suposición técnica de que la instrucción w no tiene arcos de entrada de juntura, lo cual garantiza que G' sigue siendo un DAG.



Árbol de generación con arcos deque

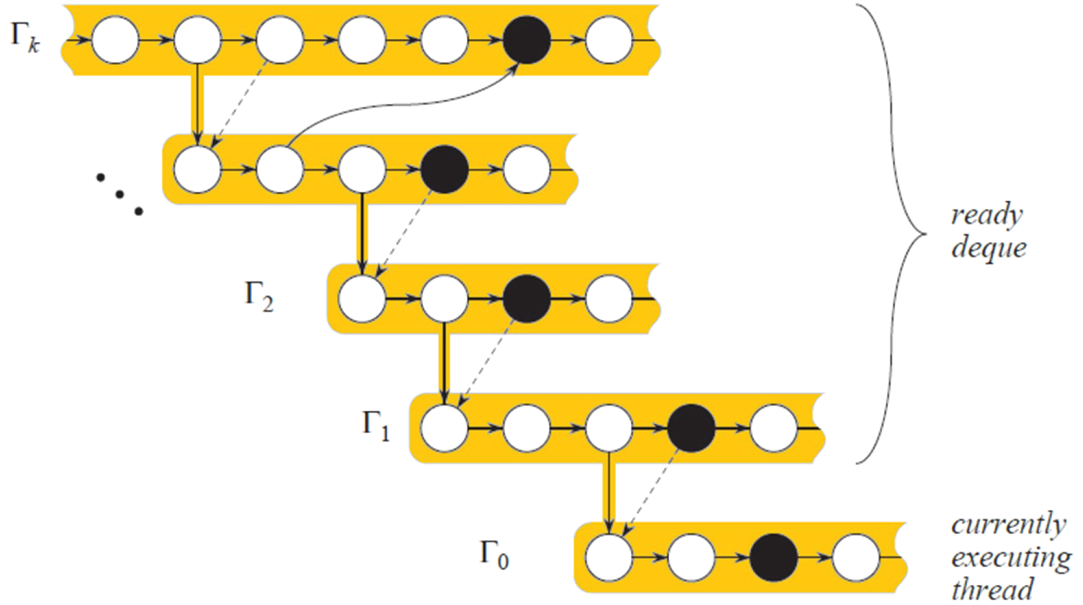


Círculo rojo: instrucción w que continúa tras un *spawn*;
Flecha roja: arco deque

Grafo extendido



- Si T_∞ es la longitud del camino más largo en G, entonces la longitud del camino más largo en G' es a lo sumo $2T_\infty$.
  - Cabe señalar que G' se utiliza únicamente como una herramienta analítica.
- Los arcos deque no tienen ningún efecto sobre la planificación o ejecución de la computación bajo el algoritmo de Work Stealing.



Grafo extendido



- Los arcos deque son la clave para definir las instrucciones críticas.
- En cualquier paso de tiempo durante la ejecución, decimos que una instrucción no ejecutada v es crítica si todas las instrucciones que la preceden (ya sea directa o indirectamente) en G' ya han sido ejecutadas; es decir, si para toda instrucción w tal que existe un camino dirigido desde w hasta v en G', la instrucción w ha sido ejecutada.

Grafo extendido



- Una instrucción crítica debe estar en estado *ready*, ya que G' contiene todos los arcos del DAG original G. Sin embargo, una instrucción *ready* puede o no ser crítica.
- Intuitivamente, las propiedades estructurales de una *ready deque*, enumeradas en el Lema 4, garantizan que si un hilo se encuentra profundo dentro de una *ready deque*, entonces su instrucción actual no puede ser crítica, porque el predecesor de esa instrucción (a través de un arco deque) todavía no ha sido ejecutado.

Definición 8. Secuencia de retraso



- Una secuencia de retraso (*delay sequence*) es una tripleta (U, R, Π) que satisface las siguientes condiciones:
- $-\ U\ =\ (u_1, u_2, u_3, \ldots, u_L)$ es un camino dirigido maximal en G'. Específicamente, para $i = 1,2, \ldots, L-1$, el arco (u_i, u_{i+1}) pertenece a G'; la instrucción u_1 no tiene arcos entrantes en G' (es decir, u_1 debe ser la primera instrucción del hilo raíz); y la instrucción u_L no tiene arcos salientes en G' (por lo tanto, u_L debe ser la última instrucción del hilo raíz).
- $-\ R$ es un número entero positivo que representa la cantidad de rondas de intentos de robo (*steal-attempt rounds*).
- $-\ \Pi\ =\ (\pi_1, \pi'_1, \pi_2, \pi'_2, \pi_3, \pi'_3, \ldots \pi_L, \pi'_L)$ es una partición de R, es decir: $R = \sum_{i=1}^{L}(\pi_i + \pi'_i)$, tal que cada $\pi'_i \in \{0, 1\}$ para $i\ =\ 1,2, \ldots, L$.

Definición 8. Secuencia de retraso



- Otros componentes
- L: longitud de la secuencia de retraso, es decir, la cantidad de instrucciones críticas analizadas.
- π_i: número de rondas asociadas directamente con la instrucción crítica u_i.
- π'_i: puede ser 0 o 1, y representa una posible ronda adicional "de transición" entre instrucciones.

Definición 8. Secuencia de retraso



- La partición Π induce una partición de una secuencia de R rondas de la siguiente manera:
  - La primera parte de la partición corresponde a las primeras π_1 rondas.
  - La segunda parte corresponde a las siguientes π'_1 rondas consecutivas después de las π_1 anteriores.
  - La tercera parte corresponde a las siguientes π_2 rondas consecutivas después de las primeras $(\pi_1 + \pi'_1)$ rondas, y así sucesivamente.
- Nos interesan principalmente las partes correspondientes a las π_i, no a las π'_i. Por lo tanto, definimos el *i*-ésimo grupo de rondas como las π_i rondas consecutivas que comienzan después de la ronda r_i, donde: $r_i = \sum_{j=1}^{i-1} (\pi_j + \pi'_j)$

Definición 8. Secuencia de retraso



- Dado que Π es una partición de R y que cada $\pi'_i \in \{0,1\}$, para todo $i = 1,2,\ldots,L$ se cumple que...

$$\sum_{i=1}^{L} \pi_i \geq R - L$$

La suma total de las rondas efectivamente asignadas a instrucciones críticas (las π_i) es al menos el total de rondas R, menos como máximo una ronda por cada instrucción, es decir, L.

- La mayoría de las rondas están directamente relacionadas con instrucciones críticas, y solo unas pocas (a lo sumo L) son intermedias o de transición. Esta propiedad es clave para mostrar que la cantidad de rondas útiles está bien acotada, lo que permite demostrar límites en el tiempo total de ejecución.

Ejemplo Secuencia de retraso



- Longitud de la secuencia: L = 3
- Total de rondas: R=10
- Queremos construir una partición $\Pi = (\pi_1, \pi'1, \pi_2, \pi'2, \pi_3, \pi'3)$ tal que:

$$\sum_{i=1}^{3}(\pi_i + \pi'_i) = 10 \quad \text{y} \quad \pi'_i \in \{0, 1\}$$

i	π_i	π'_i
1	3	1
2	4	0
3	2	0
	Total	**1**

$$\sum \pi_i = 3 + 4 + 2 = 9$$

$$\sum \pi'_i = 1$$

Total: $9 + 1 = 10 = R$

$$\sum_{i=1}^{3} \pi_i = 9 \geq R - L = 10 - 3 = 7$$

Se cumple $9 \geq 7$

- De las 10 rondas, 9 están directamente asociadas con instrucciones críticas.
- Solo 1 ronda es “intermedia” o de transición (π'_1).
- Por tanto, la mayor parte de las rondas de robo se explica por la secuencia crítica, que permite acotar el retardo total de manera eficiente.

Ejemplo Secuencia de retraso



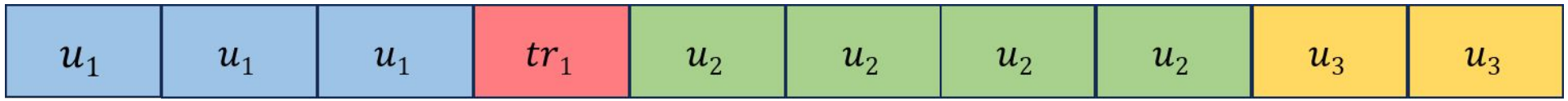
Partición de 10 rondas
Azul (u_1): 3 rondas asociadas a la primera instrucción crítica.
Rojo (tr_1): 1 ronda de transición después de u_1.
Verde (u_2): 4 rondas asociadas a la segunda instrucción crítica.
Naranja (u_3): 2 rondas asociadas a la tercera instrucción crítica.

i	π_i	π'_i
1	3	1
2	4	0
3	2	0
	Total	1

$$\sum \pi_i = 3 + 4 + 2 = 9$$

$$\sum \pi'_i = 1$$

Total: $9 + 1 = 10 = R$



Rondas de intento de robo (*índice de ronda*)

De las 10 rondas, 9 están directamente asociadas con instrucciones críticas.
Solo 1 ronda es "intermedia" o de transición (π'_1).
Por tanto, la mayor parte de las rondas de robo se explica por la secuencia crítica, que permite acotar el retardo total de manera eficiente.

Definición 8. Secuencia de retraso



- Decimos que una ronda dada de intentos de robo ocurre mientras la instrucción v es crítica si todos los intentos de robo que conforman esa ronda se inician en pasos de tiempo en los que v es crítica: En otras palabras, v debe ser crítica durante toda la ronda.
- Se dice que una secuencia de retraso (U, R, Π) ocurre durante una ejecución si, para cada $i = 1,2, \ldots, L$, todas las π_i rondas en el grupo i-ésimo ocurren mientras la instrucción u_i es crítica. Es decir, u_i debe ser crítica a lo largo de las π_i rondas.

Definición 8. Secuencia de retraso



- El siguiente lema establece que si ocurren al menos R rondas durante una ejecución, entonces alguna secuencia de retraso (U, R, Π) debe ocurrir.
- En particular, si consideramos cualquier ejecución en la que se produzcan al menos R rondas, entonces podemos identificar:
  - Un camino $U = (u_1, u_2, \ldots, uL)$ en el Grafo extendido G', y
  - Una partición $\Pi = (\pi_1, \pi'1, \pi_2, \pi'2, \ldots, \pi_L, \pi'_L)$ de las primeras R rondas, tal que, para cada $i = 1,2, \ldots, L$, todas las π_i rondas del grupo i ocurren mientras u_i es crítica.
  - Cada π'_i indica si u_i es crítica al comienzo de una ronda, pero se ejecuta antes de que la ronda termine.
  - Una ronda de este tipo no puede formar parte de ningún grupo, ya que ninguna instrucción es crítica durante toda su duración.

Dólares en el ***steal bucket***



- **Lema 9:** Considere la ejecución de una computación multihilo completamente estricta con un span de T_∞ utilizando el algoritmo de Work-Stealing en un computador con P procesadores. Si al menos $4PR$ intentos de robo ocurren durante la ejecución, entonces debe ocurrir alguna secuencia de retardo [*delay sequence*] (U, R, Π).

Si se ven **muchos intentos de robo**, eso solo puede pasar porque el avance real de la computación por su zona crítica está siendo retrasado una y otra vez.
El lema no dice todavía que eso sea improbable; solo dice que **muchos robos implican la existencia de una secuencia de demoras**.
Más adelante el artículo usa este hecho para demostrar que esas *delay sequences* son improbables y, por tanto, que también es improbable que haya demasiados robos.

Idea de prueba en la siguiente lámina.

Lema 9: idea de la prueba



- **Paso 1: contar rondas.** Cada ronda tiene menos de 4P intentos, así que al menos 4PR intentos implican al menos R rondas.
- **Paso 2: construir U hacia atrás.** Se parte de la última instrucción del hilo raíz. Se salta al predecesor en G′ que se ejecutó **más tarde**, y se repite hasta llegar a la primera instrucción. Resultado: en **cada** paso de tiempo, alguna instrucción de U es crítica.
- **Paso 3: repartir las rondas.**
  - Las rondas completas que ocurren mientras u_i es crítica van al grupo i (π_i).
  - A lo sumo una ronda "cabalga" sobre el momento en que u_i se ejecuta ($\pi'_i = 1$).

Analogía: una carrera de relevos. En todo instante, algún corredor de U tiene el testigo, y cada ronda de robos se le carga a quien lo tenía.

Dólares en el ***steal bucket***



- **Lema 10:** En cada paso, durante la ejecución de una computación multihilo completamente estricta utilizando el algoritmo de Work-Stealing, cada instrucción crítica es la instrucción *ready* de un hilo que tiene **como máximo 1 hilo** por encima (hacia el *top*) en la *ready deque* de su procesador.

Si una instrucción es crítica, entonces está en una parte de la computación que realmente importa para terminar pronto. El lema afirma que el scheduler de **work stealing** en una computación completamente estricta mantiene esa instrucción **muy cerca del "top"** de la ready deque del procesador correspondiente.
Por eso, esa instrucción crítica tiene buenas probabilidades de ejecutarse pronto o de quedar disponible para ser robada pronto.

Idea de prueba en la siguiente lámina.

Del Lema 10 al 11: dos robos bastan



- Sea v una instrucción crítica del hilo Γ, que está en la deque del procesador p.
- Por el Lema 10, hay a lo sumo un hilo Γ' encima de Γ.
  - **Caso 1**, Γ está en el top (o p lo está ejecutando): el primer robo dirigido a p se lleva Γ, o p lo sigue ejecutando. En ambos casos v se ejecuta.
  - **Caso 2**, Γ' está encima: el primer robo se lleva Γ' y el segundo se lleva Γ. Entonces v se ejecuta.
- ***Conclusión:*** mientras v siga crítica, a lo sumo un intento de robo pudo haber elegido a p.
- Como los ladrones eligen víctima al azar, que casi nadie elija a p durante muchas rondas es improbable. Eso es lo que cuantifica el Lema 11.

Dólares en el ***steal bucket***



- **Lema 11:** Considere la ejecución de cualquier computación multihilo completamente estricta con un span de T_∞ utilizando el algoritmo de Work-Stealing en un computador paralelo con $P \geq 2$ procesadores. Para cualquier instrucción v y para cualquier número $r \geq 2$ de **Rondas** de intentos de robo, la probabilidad de que cualquier conjunto particular de r Rondas ocurra mientras la instrucción v es crítica es como máximo la probabilidad de que solo 0 ó 1 intentos de robo iniciados en las primeras $r - 1$ de estas rondas seleccionen el procesador de la instrucción v, que es como máximo e^{-2r+3}

Una instrucción crítica no "atasca" el camino crítico durante muchas Rondas, salvo en un evento muy improbable. El lema muestra que secuencias largas de demora sobre el camino crítico tienen probabilidad muy pequeña.

Idea de prueba en la siguiente lámina.

Lema 11: la cuenta



- En $r - 1$ rondas hay al menos $n = 3P(r - 1)$ intentos de robo.
- Cada intento elige a p con probabilidad $1/P$, de forma independiente.
- El número esperado de intentos que eligen a p es

$$n \cdot (1/P) = \mathbf{3(r-1)}.$$

- La probabilidad de 0 o 1 acierto es:

$$(1 - 1/P)^{n} + n \cdot (1/P) \cdot (1 - 1/P)^{n-1} \leq (6r - 5) \cdot e^{-3(r-1)} \leq e^{-2r+3}$$

r (rondas)	
2	0,37
3	0,050
5	0,00091
10	4×10^{-8}

Cada ronda adicional divide la cota por $e^2 \approx 7{,}4$.

Se usan $r - 1$ rondas y no r porque la última ronda dura al menos 4 pasos. Eso garantiza que v todavía no se ha ejecutado cuando termina el último intento contado.

Volvamos al ejemplo de L = 3, R = 10



- Las rondas de distintos grupos son independientes, así que las probabilidades se multiplican.
- Con $\pi = (3, 4, 2)$: $e^{-3} \cdot e^{-5} \cdot e^{-1} = e^{-9} \approx \mathbf{0,00012}$.
- La cota general del artículo da $e^{-2R+7L} = e^{-20+21} = \boldsymbol{e} \approx \mathbf{2,7}$, es decir, ¡no dice nada!
- **Lección:** la cota general solo sirve cuando R es bastante mayor que L. Como $L \leq 2T_\infty$, se necesita R del orden de T_∞.
- Por eso el resultado final es $\boldsymbol{O(PT_\infty)}$ robos.

Dólares en el ***steal bucket***



- **Lema 12:** Considere la ejecución de cualquier computación multihilo completamente estricta con un span de T_∞ utilizando el algoritmo de Work-Stealing en un computador paralelo con P procesadores. Para cualquier $\epsilon > 0$, con probabilidad de al menos $1 - \epsilon$, ocurrirán como máximo $O(P(T_\infty + \lg(1/\epsilon)))$ intentos de robo de trabajo [*work-steal*]. El número esperado de intentos de robo es $O(P(T_\infty))$. En otras palabras, con probabilidad de al menos $1 - \epsilon$, la ejecución termina con como máximo $O(P(T_\infty + \lg(1/\epsilon)))$ dólares en el balde Steal y el número esperado de dólares en este balde es de $O(P(T_\infty))$.

El planificador de work stealing no desperdicia demasiado tiempo robando: el costo total de robar trabajo es, esencialmente, **lineal en** P **y en** T_∞ ,con un pequeño margen adicional si se desea una garantía probabilística más fuerte.

Idea de prueba en la siguiente lámina.

Lema 12: cota de la unión



- Por el Lema 9:
- $\Pr[\geq \boldsymbol{4PR\ robos}] \leq \Pr[\boldsymbol{ocurre\ alguna\ secuencia\ de\ retraso}]$
  $\leq (\boldsymbol{número\ de\ secuencias}) \times (\boldsymbol{probabilidad\ de\ cada\ una})$

Factor	Cota	De dónde sale
Caminos U		
Particiones Π		R repartido en 2L piezas
Probabilidad de una secuencia		

¿Por qué el producto es pequeño?



- Multiplicando los tres factores: $[(d+1)^2 \cdot 16 \cdot e^{14}]^{T_\infty} \cdot (\mathbf{2}/\boldsymbol{e}^2)^{\boldsymbol{R}}$
- El primer factor es “grande pero fijo” por cada unidad de T_∞.
- El segundo factor **decrece de manera exponencial**, porque

$$2/e^2 \approx 0{,}27 < 1.$$

- Se elige $R = c \cdot T_\infty + \lg(1/\varepsilon)$:
  - La parte $c \cdot T_\infty$ anula el primer factor si c es suficientemente grande.
  - La parte $\lg(1/\varepsilon)$ aporta un factor $\leq \varepsilon$.
- **Conclusión:** con probabilidad $\geq 1 - \varepsilon$, hay menos de

$4PR = O(P(T_\infty + \lg(1/\varepsilon)))$ robos.

- Como la cola es exponencial, $\boldsymbol{E[robos] = O(PT_\infty)}$.

Teorema 13

- ***Teorema 13:*** Considere la ejecución de cualquier computación multihilo completamente estricta con un work de T_1 y un span de T_∞ utilizando el algoritmo de Work-Stealing en un computador paralelo con P procesadores. Entonces, el tiempo de ejecución esperado, incluyendo el tiempo adicional de planificación (*scheduling overhead*), es $T_1/P + O(T_\infty)$. Además, para cualquier $\epsilon > 0$, con probabilidad de al menos $1 - \epsilon$, el tiempo de ejecución para P procesadores es $T_1/P + O(T_\infty + \lg P + \lg(1/\epsilon)) *$
  - * *Con la cota de Plaxton para el lema 6, este límite llega a ser $T_1/P + O(T_\infty)$, donde $1/\epsilon$ es a lo sumo polinomial en M y P.*

El tiempo de ejecución con el algoritmo de work stealing es "trabajo repartido entre P" más "costo del camino crítico", y eso vale tanto para la esperanza como, con alta probabilidad, en la ejecución real.

Dólares en el wait bucket

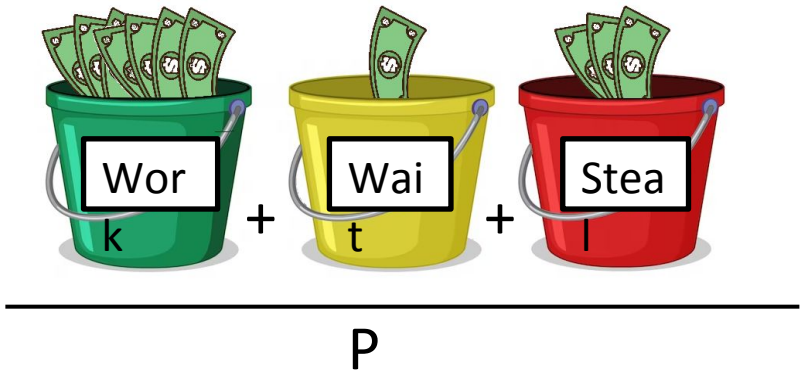


- Los lemas 7 y 12 acotan los dólares en el WORK bucket y en el STEAL bucket, así que ahora debemos acotar los dólares en el WAIT bucket.
- Esta cota está dada por el lema 6 que acota el retardo total –es decir, son los dólares del **WAIT** bucket, acotados en función de M = dólares del STEAL bucket.

Dólares en el wait bucket



- El lema 6 dice que para cualquier $\epsilon > 0$, con una probabilidad como mínimo de $1 - \epsilon$, la cantidad de dólares en el WAIT Bucket es como máximo una constante multiplicada por la cantidad de dólares en el STEAL Bucket más $O(P \lg P + P \lg(1/\epsilon))$, y la cantidad esperada de dólares en el WAIT bucket es como máximo la cantidad en el STEAL bucket.
- Ahora sumamos los dólares en los tres buckets y dividimos por P para completar esta prueba.



Teorema 13: juntando los tres baldes

- **En esperanza:**
  - $P \cdot E[T] = T_1 + E[STEAL] + E[WAIT] \leq T_1 + O(PT_\infty) + O(PT\infty)$
  - $\Longrightarrow \boldsymbol{E[T] \leq T_1/P + O(T_\infty)}$
- **Con probabilidad $\geq 1 - \varepsilon$:**
  - $STEAL = M = O(P(T_\infty + \lg(1/\varepsilon)))$
  - $WAIT = O(M + P \lg P + P \lg(1/\varepsilon))$
  - $\Longrightarrow \boldsymbol{T \leq T_1/P + O(T_\infty + \lg P + \lg(1/\varepsilon))}$

Comparación:

- El planificador *greedy* ideal es centralizado y no cuenta su propio costo: $T_1/P + T_\infty$.
- *Work stealing* es distribuido e **incluye** el costo de planificar: $T_1/P + O(T_\infty)$.
- El precio de descentralizar es solo una constante sobre T_∞.

Para la versión de alta probabilidad se reparte ε entre los dos eventos ($\varepsilon/2$ para el Lema 12 y $\varepsilon/2$ para el Lema 6).

Siguiente teorema

- El siguiente teorema acota la cantidad total de comunicación que una computación multihilo ejecutada por el algoritmo de Work-Stealing realiza en un modelo distribuido.
- El análisis asume que como máximo una cantidad constante de bytes necesita ser comunicada sobre un arco de unión para resolver la dependencia.



Teorema 14

- ***Teorema 14:*** Considere la ejecución de cualquier computación multihilo completamente estricta con un span de T_∞ utilizando el algoritmo de Work-Stealing en un computador paralelo con P procesadores. Entonces, el número total de bytes comunicados tiene una esperanza de $O(PT_\infty\,(1+nd)\,S_{max})$ donde n_d es el número máximo de arcos de juntura desde un hilo a su padre y S_{max} es el tamaño en bytes del $activation\ frame$ más grande en la computación. Además para cualquier $\epsilon > 0$, la probabilidad es al menos $1-\epsilon$ de que la comunicación total incurrida sea de

$$O(P(T_\infty + \lg(1/\epsilon))(1+n_d)S_{max})$$

Teorema 14

- El Teorema 14 indica que si se ejecuta una computación completamente estricta con work stealing en P procesadores, la cantidad total de datos que los procesadores tienen que intercambiar permanece acotada.

- El **costo de comunicación** del scheduler no explota aunque haya robos de trabajo, por el contrario, queda controlado por cuatro factores:

1. La cantidad de procesadores que participan (P)
2. La longitud del camino crítico (T_∞)
3. Cuánta sincronización entre padre-hijo puede haber ($1 + n_d$)
4. Cuánto "pesa" el frame más grande ($S_{\max}$).

Teorema 14

- La prueba que sigue resume tres fuentes principales de comunicación:

1. cuando **se roba un hilo**, mover ese trabajo cuesta como mucho S_{max} bytes.
2. Si un arco de unión entra a un nodo padre que fue robado, hay aquí un costo adicional para resolver esa dependencia.
3. Cuando un hijo habilita a su padre y lo pone en la *ready deque* del procesador del hijo, también puede haber comunicación adicional, hasta $n_d\, S_{max}$ por el robo.

- En resumen, el Teorema 14 afirma que, en computaciones completamente estrictas, la comunicación total provocada por work stealing es proporcional al número de procesadores, al camino crítico, al tamaño del frame más grande y al grado de sincronización padre-hijo; además, esa cota vale tanto en esperanza como con alta probabilidad.

Prueba teorema 14

- Probamos la cota (límite) para la expectativa.
- La cota con alta probabilidad es similar.
- Con base en nuestro argumento de los baldes, la cantidad esperada de intentos de robo es $O(PT_\infty)$.
- La comunicación también ocurre siempre que un arco de unión incide en un hilo padre con origen en uno de sus hijos y el padre fue "robado", pero como cada arco de unión es responsable de máximo una cantidad constante de bytes, el costo de comunicación en el que se incurre es como máximo $O(n_d)$ por cada robo.

Prueba teorema 14

- Finalmente, podemos tener comunicación cuando un hilo hijo habilita (*enable*) a su padre y coloca al padre en la *ready deque* del procesador que estaba ejecutando el hijo.
- Este evento puede ocurrir como máximo n_d veces para cada ocasión en que el padre es robado, de tal manera que el costo en que se incurre por la comunicación es como máximo $n_d S_{max}$ por cada robo.
- De esta manera el costo total esperado en comunicación es

$$O(PT_\infty (1 + n_d) S_{max})$$

Prueba teorema 14

- Los límites de la comunicación en este teorema son existencialmente ajustados en el sentido de que existen computaciones totalmente estrictas que requieren $\Omega(PT_\infty\,(1+n_d)\,S_{max})$ comunicaciones en total para cualquier planificación de ejecución que consigue un speedup lineal.
- Este resultado se obtiene de manera directa a partir de un teorema de Wu y Kung (1991), quienes demostraron que las computaciones de divide y vencerás [un caso especial de las computaciones completamente estrictas con $n_d = 1$] requieren toda esa comunicación.

Prueba teorema 14

- En el caso con $n_d = O(1)$ y cuando el algoritmo alcance el speedup lineal esperado – es decir, cuando $P = O(T_1/T_\infty)$ – el costo de la comunicación total será como máximo $O(T_1 S_{max})$, que confirma lo que dice la sabiduría popular: que los algoritmos de work-stealing requieren mucha menos comunicación que las posibles $\Theta(T_1 S_{max})$ comunicaciones de los algoritmos de work-sharing.

Del papel a la práctica

- **Cilk (MIT):** lenguaje basado en C cuyo *runtime* usa este algoritmo. Empíricamente, $T_1/P + T_\infty$ predice bien el desempeño real.
- **Aplicaciones:** ajedrez (★Socrates, Cilkchess), plegamiento de proteínas, renderizado y búsqueda con retroceso.
- **Cilk-NOW:** redes de estaciones de trabajo, con paralelismo adaptativo y tolerancia a fallos transparente.
- **Arora, Blumofe y Plaxton (1998):** extienden la cota de tiempo $O(T_1/P + T_\infty)$ a computaciones **arbitrarias** (no necesariamente estrictas). Usan una deque **no bloqueante** y admiten procesadores que entran y salen (multiprogramación). Esto se implementó en la biblioteca Hood.
- Esa línea de deques no bloqueantes es la que llega a Java mediante el *framework* Fork/Join de Doug Lea.

Conclusiones

- **Problema:** planificar computaciones multihilo dinámicas de forma descentralizada.
- **Algoritmo:** una deque por procesador, y ladrones aleatorios que roban por el *top*.
- **Resultados para computaciones completamente estrictas:**
  - Tiempo esperado: $T_1/P + O(T_\infty)$.
  - Espacio: $\leq S_1 P$.
  - Comunicación esperada: $O(PT_\infty (1 + n_d) S_{max})$
  - Los tres son óptimos salvo una constante.
- **Técnica:** argumento contable de tres baldes, secuencia de retraso y juego de reciclaje.
- **Consecuencia:** *speedup* lineal cuando $P = O(T_1/T_\infty)$, y comunicación mucho menor que con *work sharing*. Es la justificación formal de la "sabiduría popular".
- **Legado:** Cilk, Arora–Blumofe–Plaxton, Fork/Join de Java, Intel TBB, el planificador de Go.
- Hay abundante literatura posterior en resultados teóricos, optimización de implementaciones y adaptación a arquitecturas específicas.

Otros artículos para revisar

- **Blelloch**, G.E., **Gibbons**, P.B., **Matias**, Y.: *Provably efficient scheduling for languages with fine-grained parallelism*. J. ACM 46(2), 281–321 (1999)
  - Mejora el modelo de Blumofe en lo relacionado con las cotas de espacio para un sistema multiprocesador con memoria compartida global.
- **Fatourou**, P., **Spirakis**, P.: *Efficient scheduling of strict multithreaded computations*. Theory Comput. Syst. 33(3), 173–232 (2000)
  - Extiende el modelo de Blumofe a computaciones multi-hilo estrictos (hilo dependiente de los ancestros, no solo del padre)
- **Berenbrink**, P., **Friedetzky**, T., **Goldberg**, L.A.: *The natural work-stealing algorithm is stable*. SIAM J. Comput. 32(5), 1260–1279 (2003)
  - Muestra que el algoritmo de work stealing es estable incluso bajo distribuciones de carga grandemente desbalanceados.

Work Stealing y abstracciones de concurrencia livianas [hilos ligeros]

- Looking at Java 21: Virtual Threads
  - https://belief-driven-design.com/looking-at-java-21-virtual-threads-bd181/
- Go's work-stealing scheduler
  - https://rakyll.org/scheduler/
- How Go’s Goroutines Actually Work Under the Hood (The Scheduler Secret)
  - Ver el documento Como_funcionan_realmente_las_goroutines_de_Go_bajo_el_capo.pdf

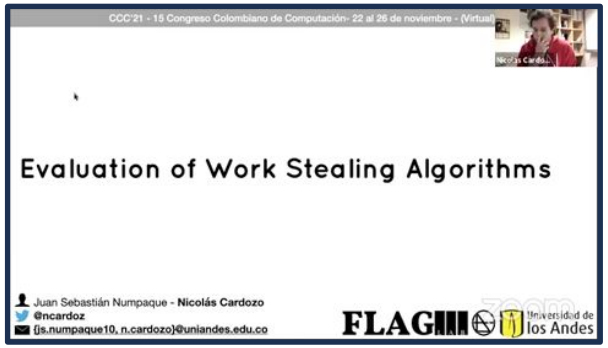
Cuidado: ¿aplican las cotas a Java y Go?

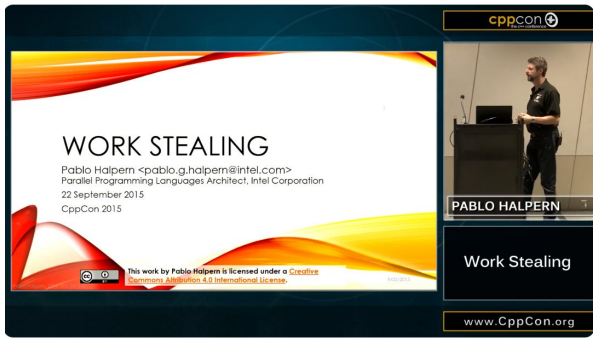
- **ForkJoinPool (Java 7+):** usa deques por hilo y los ladrones roban por el *top*. Es esencialmente el algoritmo del artículo, en su versión no bloqueante.
- **Hilos virtuales (Java 21, JEP 444):** su planificador por defecto es un ForkJoinPool en modo FIFO.
- **Go:** cada P tiene una cola local, y un P ocioso roba **la mitad** de la cola de otro.
- **Pero hay diferencias importantes:**
  - Hilos virtuales y *goroutines* se bloquean en E/S, *locks* y canales arbitrarios, así que no son computaciones completamente estrictas.
  - El modo FIFO y el robo de la mitad cambian el algoritmo.
  - Por eso **no** están garantizadas las cotas $T_1/P + O(T_\infty)$ ni $S_1 P$.
  - Lo que sí se conserva es la idea: descentralizar y robar solo cuando hay ocio.

Más material para revisar

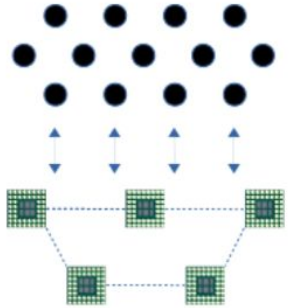
- [CCC'21] Evaluation of work stealing algorithms
- https://www.youtube.com/watch?v=pmheQLUNZaw

- CppCon 2015: Pablo Halpern “Work Stealing"
- https://www.youtube.com/watch?v=iLHNF7SgVN4





Work Stealing Simulator



- https://wssimulator.github.io/index.html
- WS-Simulator se utiliza para ejecutar una aplicación (una lista de tareas con o sin dependencias) en una plataforma multiprocesador conectada mediante una topología específica.
- Permite experimentar con diferentes algoritmos de Work Stealing, diferentes topologías, diferentes estrategias de robo y diferentes tipos de aplicaciones.
- Genera una cantidad suficiente de registros para un análisis detallado de cada escenario.

Referencia principal

- Robert D. Blumofe and Charles E. Leiserson. Scheduling multithreaded computations by work stealing. *Journal of the ACM*, 46(5):720–748, 1999.

Gracias